

Module 4

DataSources

Connect Data Sources

Unified Data Graph (UDG) – Datasource Overview

- **Datasources power your UDG and its schema**
 - **Can be attached to any field in the UDG schema**
 - **Support nested configuration**
- **Quick Start vs Full Control**
 - **You can add datasources directly to UDG without registering them as APIs in Tyk**
 - **For advanced features (rate limits, quotas, transformations), import the API into Tyk first**
- **Supported Datasource Types:**
 - **REST**
 - **GraphQL**
 - **SOAP (via REST Datasource)**
 - **Kafka**

GraphQL Datasource Overview

- Makes GraphQL queries to upstream GraphQL services
- Configured similarly to REST Datasource
- Handles query and response specifics automatically

Schema Example:

```
type Query {  
  employee(id: Int!):  
    Employee  
}  
type Employee {  
  id: Int!  
  name: String!  
}
```

Example Query:

```
query TykCEO {  
  employee(id: 1) {  
    id  
    name  
  }  
}
```

GraphQL Datasource Overview

- **GraphQL Response Parsing**

Actual Response:

```
{
  "data": {
    "employee": {
      "id": 1,
      "name": "Martin
Buhr"
    }
  }
}
```

Tyk Extracts:

```
{
  "employee": {
    "id": 1,
    "name": "Martin Buhr"
  }
}
```

GraphQL Datasource Overview

REST vs GraphQL

- REST APIs do not wrap responses
- GraphQL APIs wrap responses in root fields (like data, employee)
- GraphQL field mapping is enabled by default

Why Use GraphQL Datasource?

- Specification-compliant GraphQL requests
- Supports nested REST & GraphQL APIs in a single schema
- Query planner sends correct queries to upstreams

How the Query Planner Works

- Checks if a field has a datasource
- That datasource resolves the field and its children
- If a nested field has its own datasource, ownership shifts

GraphQL Datasource Overview

Summary

- Tyk supports full GraphQL Datasource configuration
- Automatic unwrapping of data field
- Query planner handles mixed datasources

Kafka Datasource Overview

- Enables subscribing to Kafka topics and querying events with GraphQL.
- Integrates Kafka's powerful pub/sub model into your Universal Data Graph.
- Operates based on Kafka's consumer group mechanism.

Kafka Consumer Groups

- A group of cooperating consumers that share processing load.
- Kafka automatically manages membership and rebalances partitions.
- Allows for horizontal scaling and fault tolerance.

Kafka Datasource Overview

Basic Kafka Configuration

- `broker_addresses`: Kafka broker hosts
- `topics`: Kafka topics to subscribe to
- `group_id`: Consumer group identifier
- `client_id`: Optional identifier for logging and debugging

Kafka Datasource Overview

Configuring via Tyk Dashboard

- Click the field to attach Kafka DataSource.
- In Configure Data Source, choose KAFKA.
- Provide:
 - Data source name
 - Broker addresses
 - Topics
 - Group ID
 - Client ID
 - (Optional) Kafka version, balance strategy, field mappings
- Click Save.

Kafka Datasource Overview

Configure data source

[PRODUCTINFO > DESCRIPTION](#)

Select data source type

Choose from existing Tyk APIs :

▼ GraphQL | Tyk

69

▼ REST | Tyk

18

Or, choose from data sources previously configured in this UDG :

▼ Data Sources | Tyk

Or, add a new external data source

REST

GRAPHQL

KAFKA



Kafka Datasource Overview

Once done the field you just configured will show information about data source type and name:



The screenshot displays a configuration interface with two sections: 'Type | Query' and 'Type | Subscription'. The 'Type | Query' section contains a single entry 'topProducts' with a teal dot, followed by the text 'topProducts | GraphQL'. The 'Type | Subscription' section contains a single entry 'stock' with a teal dot, followed by the text 'kafka-consumer-group | Kafka'. A teal arrow points from the 'stock' entry in the 'Type | Subscription' section to the 'kafka-consumer-group' text.

Type Query
● topProducts

Type Subscription
● stock

Kafka Datasource Overview

GraphQL Subscriptions with Kafka

Real-Time Subscriptions

Use GraphQL subscription queries to listen to Kafka events.

Define Subscription type in your schema:

```
type Product {  
  name: String  
  price: Int  
  inStock: Int  
}  
  
type Subscription {  
  productUpdated: Product  
}
```

Subscribers will get real-time updates on every event.

Example Subscription Query

```
subscription Products {  
  productUpdated {  
    name  
    price  
    inStock  
  }  
}
```

- Automatically receives new product data when Kafka publishes an event.
- Compatible with most GraphQL clients.

Kafka Datasource Overview

Publishing Test Events

Simulating Kafka Events

Publish event to product-updates topic:

```
{
  "productUpdated": {
    "name": "product1",
    "price": 1624,
    "inStock": 219
  }
}
```

- Use any Kafka producer or GUI tool.
- Useful for testing subscriptions and schema mappings.

Kafka Datasource Overview

Summary

Kafka DataSource in Tyk

- Seamlessly integrates Kafka with GraphQL APIs.
- Allows real-time streaming via GraphQL subscriptions.
- Scalable via Kafka consumer groups.
- Easy setup via Tyk Dashboard or API Definition.

REST Datasource Overview

- REST DataSources connect REST APIs to your GraphQL schema
- Enables field-level resolution via REST endpoints
- Supported for both external APIs and Tyk-managed APIs

REST Datasource Overview

- Select the field in your schema
- Configure as External REST DataSource
 - Name
 - URL
 - Method (GET, POST, etc.)
 - Optional: Headers, Field Mapping

Configure data source

QUERY > GETACTIVEUSERS

Data source details

Data source name

getActiveUsers_Query

URL

Enter data source url

Method

Select method type for your data source

☒ Add headers

ADD

Headers

HEADER KEY	HEADER VALUE	DELETE
Enter header key	Enter header value	

Field mapping

☐ Use default field mapping

Path

Enter path

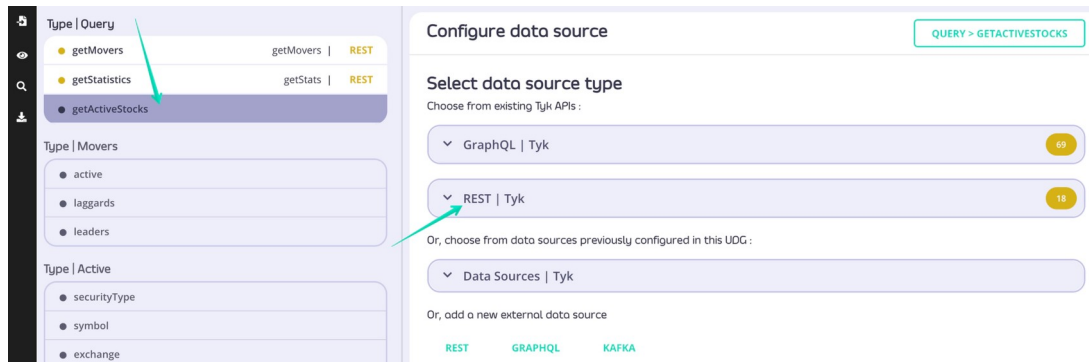
RESET

SAVE AND UPDATE API

REST Datasource Overview

Attaching Tyk REST API

- Select the field
- Choose REST | Tyk from the dropdown
- Pick from existing Tyk APIs
- Provide endpoint and method
- Optional headers and mapping



Save and Generate Resolver

- Click "Save & Update API"
- Resolver is auto-generated
- GraphQL runtime will call the REST endpoint when the field is queried

REST Datasource Overview

Auto-generate REST DataSource via OAS

- Use Tyk Dashboard API
- Endpoint: `POST /api/data-graphs/data-sources/import`
- Request Body:

```
{  
  "type": "string",  
  "data": "string"  
}
```

`type` is an enum with the following possible values:

- `openapi`
- `Asyncapi`

To import an OAS specification you need to choose `openapi`

REST Datasource Overview

Summary

- REST DataSources make it easy to integrate legacy REST APIs
- Both external and internal APIs are supported
- Automatic configuration possible with OAS